

I. CƠ SỞ LÝ THUYẾT

1.1. Tổng quan về Dịch ngược phần mềm (Reverse Engineering)

1.1.1. Tìm hiểu về Dịch ngược

Kỹ thuật dịch ngược (Reverse Engineering - RE) là một kỹ thuật đi ngược lại quá trình chế tạo để tìm hiểu cơ chế hoạt động, thiết kế và kiến trúc của một chương trình phần mềm.

Vai trò: Quá trình này đặc biệt cần thiết và phát huy tác dụng tối đa trong các trường hợp khi người phân tích không có thông tin về mã nguồn gốc, hoặc khi các thông tin này đã bị mất, bị phá hủy. Bằng cách phân tích các tệp thực thi (như PE file trên Windows), người ta có thể hiểu được cách phần mềm hoạt động, tương tác với hệ điều hành và sử dụng tài nguyên máy tính

1.1.2. Trình Biên dịch và Phân dịch

Đây là hai quá trình có chiều hoạt động hoàn toàn trái ngược nhau

Trình biên dịch (Compiler): Đây là một chương trình đảm nhận việc chuyển đổi các chuỗi câu lệnh được viết bằng ngôn ngữ lập trình bậc cao (như Java, C, C++,...) thành ngôn ngữ máy. Mục đích của việc này là đưa mã nguồn về dạng nhị phân cơ bản nhất để máy tính có thể trực tiếp hiểu và thực thi.

Trình phân dịch (Disassembler): Ngược lại với quá trình biên dịch, trình phân dịch là một chương trình thực hiện công việc phân tích và chuyển đổi các tệp tin thực thi từ dạng mã máy quay trở lại thành hợp ngữ (Assembly). Vì con người không thể đọc trực tiếp mã máy, việc dịch ngược ra hợp ngữ giúp các nhà phân tích nhìn thấy các tập lệnh cơ bản thao tác trên thanh ghi và bộ nhớ (ví dụ như lệnh mov, cmp, xor,...) để tái tạo lại logic của chương trình ban đầu. Các công cụ Disassembler nổi tiếng thường được sử dụng bao gồm IDA Pro, Ghidra hoặc Radare2.

1.2. Định dạng tệp thực thi PE và Kiến trúc hệ thống

1.2.1. Cấu trúc tệp PE (Portable Executable)

PE (Portable Executable) là định dạng tệp thực thi tiêu chuẩn và riêng biệt của hệ điều hành Windows. Định dạng này không chỉ áp dụng cho tệp .exe mà còn bao gồm các tệp .dll, .com, .net, .cpl, .clf. Tệp bản thân tệp PE có thể sử dụng tài nguyên của máy tính để chạy và tạo thành một tiến trình (process).

Cấu trúc cốt lõi của một tệp PE bao gồm:

- PE Header: Phần đầu của tệp chứa các thông tin nhận dạng và sơ đồ bố trí vật lý/logic. Nó bao gồm chữ ký MZ (dấu hiệu nhận biết tệp thực thi DOS cũ) và chữ ký PE (50 45 00 00) để hệ điều hành nhận diện đây là tệp PE hợp lệ.
- ImageBase: Đây là địa chỉ bộ nhớ được ưu tiên để hệ điều hành nạp (load) tệp PE vào không gian địa chỉ ảo. Trong 99% các trường hợp, giá trị mặc định của ImageBase là 0x400000 (hoặc 400000h). Trình nạp (PE Loader) sẽ cố gắng nạp tệp tại địa chỉ này, trừ khi vùng nhớ đó đã bị chiếm dụng bởi một module khác.
- Section (Phân vùng): Tệp PE được chia thành nhiều phần (section) chứa mã lệnh (.text), dữ liệu (.data), v.v. Khi ánh xạ vào bộ nhớ, mỗi section phải tuân thủ sự liên kết (Section Alignment) và bắt đầu tại một địa chỉ là bội số của giá trị alignment (thường là 0x1000 - tương đương 4096 bytes, hoặc 0x10000 - 64KB). Ví dụ: Nếu section đầu tiên nạp ở 401000h có kích thước 10 bytes, thì section tiếp theo buộc phải bắt đầu từ 402000h.
- Entry Point (Điểm bắt đầu): Nằm trong phần OptionalHeader, AddressOfEntryPoint lưu địa chỉ của câu lệnh đầu tiên sẽ được thực thi khi chương trình khởi chạy.

1.2.2. Bảng Import (IAT) và Windows API

Ứng dụng ở không gian người dùng (User Mode) không thể tương tác trực tiếp với phần cứng hay vùng nhớ lõi (Kernel). Để làm được điều này, chúng phải gọi các hàm Windows API do hệ điều hành cung cấp. Bảng Import (Import Address Table - IAT) trong cấu trúc PE lưu trữ danh sách các hàm API này cùng với địa chỉ thực tế của chúng trong bộ nhớ.

Các hàm API được chia thành ba nhóm thư viện (DLL) chính:

- Kernel32.dll: Chứa các API cốt lõi (Kernel APIs) để quản lý bộ nhớ, tệp tin, tiến trình và tương tác trực tiếp với hệ thống.

- User32.dll: Chứa các API cấp cao (User APIs) chuyên xử lý giao diện người dùng, hiển thị cửa sổ, tạo hộp thoại thông báo và quản lý tương tác.
- GDI32.dll: Cung cấp các hàm API (GDI APIs) xử lý đồ họa cấp thấp như vẽ đường thẳng, hiển thị pixel.

1.2.3. Tổ chức bộ nhớ và Thanh ghi (Registers)

Bộ vi xử lý sử dụng các thanh ghi để lưu trữ dữ liệu tạm thời với tốc độ truy xuất cực nhanh. Kiến trúc x86 chia thanh ghi thành 3 loại chính: thanh ghi dữ liệu, thanh ghi chỉ số và thanh ghi trạng thái.

Thanh ghi đa dụng (General Purpose Registers): Gồm 8 thanh ghi 32-bit là EAX, EBX, ECX, EDX, ESP, EBP, ESI, và EDI. Các thanh ghi này có thể được chia nhỏ để truy xuất linh hoạt. Ví dụ: EAX (32-bit) có thể chia thành AX (16-bit thấp), sau đó AX lại chia tiếp thành AH (8-bit cao) và AL (8-bit thấp).

Con trỏ lệnh (Instruction Pointer - EIP): Đây là thanh ghi đặc biệt chuyên chứa địa chỉ của câu lệnh mã máy tiếp theo mà CPU sẽ thực thi. EIP sẽ chạy tuần tự trừ khi gặp các lệnh điều hướng luồng (như lệnh nhảy).

1.2.4. Ngôn ngữ Assembly

Hợp ngữ (Assembly) là ngôn ngữ bậc thấp, ánh xạ trực tiếp thành mã máy mà CPU có thể hiểu được. Trong quá trình phân tích tĩnh hoặc gỡ rối động (nhấn F7/F8 để chạy từng lệnh), kỹ sư dịch ngược phải làm việc liên tục với các tập lệnh này:

Lệnh thao tác dữ liệu:

- MOV: Sao chép dữ liệu từ nguồn (source) sang đích (destination). Dữ liệu ở nguồn không bị thay đổi. Có đặc điểm không thể di chuyển trực tiếp dữ liệu từ vùng nhớ này sang vùng nhớ khác, và không thể ghi giá trị trực tiếp vào các thanh ghi Segment.
- XCHG: Hoán đổi giá trị giữa hai toán hạng.
- LEA (Load Effective Address): Nạp địa chỉ của bộ nhớ vào thanh ghi.

Lệnh tính toán logic: AND, OR, XOR thực hiện các phép toán bit logic tương ứng.

Lệnh so sánh và điều hướng (Flow Control):

- **CMP**: Thực hiện phép trừ giữa hai toán hạng để so sánh (không lưu kết quả mà chỉ cập nhật cờ trạng thái). Thường dùng để kiểm tra điều kiện (ví dụ: so sánh serial nhập vào với serial đúng).
- **TEST**: Tương tự CMP nhưng sử dụng phép toán AND logic.
- **JNZ / JNE** (Jump if Not Zero / Jump if Not Equal): Lệnh nhảy có điều kiện. Nếu kết quả so sánh trước đó là khác nhau (khác 0), CPU sẽ nhảy tới một đoạn mã khác (ví dụ: nhảy đến hộp thoại báo lỗi bản quyền).
- **JMP**: Lệnh nhảy vô điều kiện, ép luồng thực thi chuyển hướng ngay lập tức đến địa chỉ được chỉ định, thường được dùng trong kỹ thuật patch nhị phân để bỏ qua xác thực.

1.3. Các phương pháp phân tích bảo mật phần mềm

1.3.1. Phân tích tĩnh (Static Analysis)

Phân tích tĩnh là quá trình kiểm tra, phân tích các tệp thực thi (như file PE trên Windows) mà không hề chạy chúng. Mục tiêu là thu thập càng nhiều thông tin về cấu trúc và logic của chương trình càng tốt.

Trích xuất chuỗi (String Analysis): Khi lập trình viên viết code, họ thường để lại các chuỗi văn bản (strings) có thể đọc được. Việc trích xuất các chuỗi này (sử dụng công cụ như lệnh strings trên Linux hoặc tính năng tìm kiếm của Ghidra) cung cấp những manh mối cực kỳ quan trọng.

Ứng dụng thực tế: Người phân tích thường tìm kiếm các chuỗi thông báo lỗi như "License invalid", "Trial expired", hoặc các thông tin nhạy cảm bị "hardcode" như "admin:password123", các đường dẫn URL kết nối đến máy chủ C&C của mã độc, và tên các hàm API (ví dụ: CreateFileA, WriteProcessMemory). Từ các chuỗi này, người ta có thể tìm ra vị trí đoạn code chịu trách nhiệm kiểm tra bản quyền hoặc thực hiện hành vi độc hại.

Phân tích luồng thực thi thông qua mã giả (Pseudocode):

Mục đích: Mã máy (0 và 1) hoặc hợp ngữ (Assembly) rất khó đọc. Các công cụ dịch ngược mạnh mẽ (Decompiler) như Ghidra hoặc IDA Pro có khả năng dịch ngược mã máy thành "mã giả".

Cách thức: Nhờ mã giả, cấu trúc của chương trình trở nên rõ ràng hơn với các vòng lặp, câu lệnh điều kiện (if/else). Người phân tích có thể dễ dàng theo dõi luồng thực thi,

ví dụ như nhận diện một đoạn kiểm tra if (strcmp(serial, VALID_SERIAL) != 0) để biết chương trình đang so sánh key bản quyền. Từ đó, họ có thể dò ngược (Cross-references) để xem hàm nào đã gọi đoạn kiểm tra này.

1.3.2. Phân tích động (Dynamic Analysis)

Phân tích động là quá trình chạy chương trình trong một môi trường được kiểm soát chặt chẽ để quan sát hành vi thực tế của nó theo thời gian thực. Phương pháp này thường bù đắp cho những hạn chế của phân tích tĩnh (đặc biệt khi mã bị mã hóa hoặc nén - packed).

Sử dụng trình gỡ rối (Debugger): Các công cụ tiêu biểu như x64dbg (trên Windows) hay GDB (trên Linux) cho phép người phân tích can thiệp trực tiếp vào quá trình chạy của phần mềm.

Đặt các điểm dừng (Breakpoints): Thay vì để chương trình chạy tuốt lượt, người phân tích sẽ đặt các Breakpoints tại các vị trí nghi ngờ.

- Ví dụ: Họ có thể đặt Breakpoint ngay tại các hàm API quan trọng của hệ điều hành như CreateProcessA (để xem phần mềm có gọi chương trình khác không), MessageBoxW (khi phần mềm hiện hộp thoại báo lỗi bản quyền), hoặc tại các lệnh so sánh chuỗi (như strcmp). Khi chương trình chạy đến các điểm này, nó sẽ bị tạm dừng.

Theo dõi trạng thái bộ nhớ và thanh ghi (Registers & Memory):

- Khi chương trình tạm dừng tại Breakpoint, người phân tích có thể chạy từng bước một để xem chuyện gì đang xảy ra.
- Họ sẽ kiểm tra các thanh ghi của CPU (ví dụ: thanh ghi EAX thường chứa giá trị trả về của một hàm kiểm tra, xác định xem kết quả là đúng hay sai) và ngăn xếp (Stack) để xem các tham số nào đang được truyền vào hàm.
- Can thiệp: Tại thời điểm chạy, người gỡ rối thậm chí có thể thay đổi giá trị của các thanh ghi hoặc chỉnh sửa trực tiếp mã lệnh (Patching) – ví dụ như thay đổi một lệnh nhảy có điều kiện JNZ (Jump if Not Zero) thành một lệnh nhảy vô điều kiện JMP hoặc điền các lệnh vô nghĩa NOP để ép chương trình bỏ qua bước kiểm tra xác thực.

1.4. Kỹ thuật đánh giá và can thiệp cơ chế xác thực

Khi phân tích bảo mật hoặc dịch ngược phần mềm, việc hiểu rõ cách chương trình tự bảo vệ và xác thực bản quyền là yếu tố then chốt. Dưới đây là ba kỹ thuật can thiệp phổ biến nhất:

1.4.1. Kỹ thuật Patching mã nhị phân (Binary Patching)

Patching mã nhị phân là kỹ thuật chỉnh sửa vĩnh viễn tệp thực thi (như .exe hoặc .dll) bằng cách thay đổi trực tiếp các byte mã máy (hex) của nó. Quá trình này nhằm mục đích bẻ lái luồng thực thi mặc định của chương trình.

- Cách thức thực hiện: Thông qua quá trình phân tích động bằng trình gỡ rối (ví dụ: x64dbg), người phân tích sẽ xác định chính xác địa chỉ bộ nhớ chứa đoạn mã kiểm tra bản quyền (thường theo sau bởi một lệnh so sánh CMP).
- Thay đổi lệnh nhảy (Jump Instructions): Kỹ thuật kinh điển nhất là can thiệp vào các lệnh nhảy có điều kiện. Ví dụ, nếu chương trình sử dụng lệnh JNZ (Jump if Not Zero) để nhảy đến hộp thoại báo lỗi "Trial Expired" (Hết hạn dùng thử), người phân tích có thể sửa mã byte của lệnh JNZ thành JMP (Jump - nhảy vô điều kiện) để ép chương trình luôn đi vào nhánh xác thực thành công.
- Sử dụng NOP: Một phương pháp khác là ghi đè toàn bộ đoạn mã kiểm tra hoặc lệnh nhảy bằng mã NOP (No Operation - mã hex là 90). Khi CPU đọc đến lệnh NOP, nó sẽ không làm gì cả và đi tiếp sang lệnh tiếp theo, từ đó vô hiệu hóa hoàn toàn bước kiểm tra. Cuối cùng, tệp đã chỉnh sửa sẽ được lưu lại thành một bản vá (patched file).

1.4.2. Kỹ thuật Unpacking (Giải nén và Gỡ rối mã)

Trước khi tìm hiểu Unpacking, cần hiểu Packing là gì. Các nhà phát triển (hoặc tác giả mã độc) thường dùng kỹ thuật đóng gói để giảm kích thước tệp tin (nén) và che giấu mã lệnh (bằng cách mã hóa hoặc xáo trộn - obfuscation) nhằm làm khó quá trình dịch ngược.

Khi một chương trình bị pack, cấu trúc gốc của nó bị thay đổi: con trỏ lệnh (EIP) thay vì trỏ thẳng vào điểm bắt đầu gốc (OEP) thì lại trỏ vào một đoạn mã phụ trợ gọi là "stub". Đoạn stub này có nhiệm vụ giải nén mã gốc vào bộ nhớ khi chương trình chạy. Nếu không unpack, các công cụ phân tích sẽ chỉ đọc được đoạn mã vô nghĩa hoặc đoạn mã giải nén của stub.

Các phương pháp Unpacking:

1. Nhận diện (Identification): Bước đầu tiên là phải dùng các công cụ phân tích cấu trúc như PEiD để lấy thông tin xem tệp thực thi đang bị đóng gói bởi trình packer nào (ví dụ: UPX, ASPack, VMProtect...).
2. Unpacking tự động (Automated Unpacking): Nếu ứng dụng được bảo vệ bởi các packer phổ biến và cơ bản (như UPX), người phân tích có thể dùng các công cụ

có sẵn để bung file tự động một cách dễ dàng (ví dụ: chạy lệnh upx -d target.exe)

3. Unpacking thủ công (Manual Unpacking): Đối với các packer phức tạp hoặc tự viết (custom packer), việc giải nén tự động là bất khả thi. Kỹ sư phải tiến hành phân tích động bằng trình gỡ rối (ví dụ: x64dbg) để bắt quả tang chương trình lúc nó vừa tự giải mã xong trong bộ nhớ RAM. Quy trình kinh điển bao gồm các bước:
 - Tìm OEP (Original Entry Point): Sử dụng các kỹ thuật theo dõi thanh ghi để tìm ra khoảng khắc đoạn mã stub nhường lại quyền điều khiển cho mã gốc.
 - Dump Memory: Khi chương trình dừng lại đúng tại OEP, người phân tích sẽ trích xuất (dump) toàn bộ vùng nhớ đang chứa mã lệnh đã được giải mã rõ ràng xuống ổ cứng.
 - Fix Imports (Sửa bảng Import): Bản dump thường bị hỏng bảng IAT (Import Address Table). Kỹ sư phải dùng các công cụ/plugin hỗ trợ (như ScyllaHide) để xây dựng lại bảng này và liên kết nó với bản dump để tạo thành một tệp thực thi (unpacked file) hoàn chỉnh và có thể chạy được độc lập.

Phân tích sau Unpack: Chỉ sau khi Unpack thành công, mã nguồn mới lộ diện để kỹ sư tiếp tục ném tệp tin vào các công cụ Decompiler như IDA Pro, dnSpy, hoặc ILSpy để đọc hiểu logic xác thực thực sự bên trong.

1.4.3. Can thiệp dữ liệu cấu hình cục bộ

Nhiều ứng dụng không kiểm tra bản quyền bằng thuật toán phức tạp mà dựa vào các cờ (flags) lưu trữ cục bộ trên máy tính người dùng, chẳng hạn như trong Windows Registry hoặc các tệp cấu hình ẩn (Configuration files).

Cách thức thực hiện:

- Theo dõi hệ thống: Người phân tích sử dụng các công cụ như Process Monitor (ProcMon) để giám sát theo thời gian thực mọi hành vi đọc/ghi tệp tin và Registry của ứng dụng. Ví dụ, họ có thể lọc các tiến trình tìm kiếm đường dẫn chứa từ khóa "license" trong thư mục C:\ProgramData\... hoặc các key trong HKLM\SOFTWARE\...
- Đánh lừa cơ chế đọc (Spoofing): Bằng cách sử dụng các kỹ thuật hooking (như Frida API), người can thiệp có thể chặn lệnh gọi API đọc Registry của hệ điều hành (ví dụ: RegQueryValueExW). Khi ứng dụng truy vấn thông tin bản quyền, hook sẽ trả về một dữ liệu giả mạo (ví dụ: chuỗi "VALID_LICENSE_9999") khiến ứng dụng tin rằng thiết bị đã được cấp phép.

- Chỉnh sửa trực tiếp: Thay đổi mã định danh phần cứng (Hardware ID - HWID) trong Registry để qua mặt các cơ chế khóa ứng dụng theo thiết bị.

1.4.4 Đánh giá xác thực mạng (Network Analysis)

Đối với các phần mềm hiện đại hoặc dịch vụ đám mây, cơ chế xác thực thường yêu cầu kết nối internet để giao tiếp với máy chủ cấp phép (License Server). Việc đánh giá mạng giúp xác định cấu trúc gói tin xác thực và điểm yếu trong giao thức.

Cách thức thực hiện:

- Phân tích lưu lượng: Sử dụng công cụ bắt gói tin như Wireshark để giám sát các luồng dữ liệu mạng. Người phân tích sẽ tìm kiếm các yêu cầu HTTP/HTTPS (ví dụ: POST request chứa auth token) gửi đến các domain máy chủ cấp phép.
- Giám sát và chặn API mạng: Bằng các công cụ phân tích động, người ta có thể tìm và đặt điểm dừng tại các hàm API mạng của Windows, tiêu biểu là thư viện winhttp.dll (với các hàm như WinHttpRequest hoặc WinHttpConnect).
- Can thiệp (Interception): Sau khi xác định được API chịu trách nhiệm gửi thông tin xác thực, người phân tích có thể viết script để chặn (block) hoàn toàn yêu cầu này gửi ra ngoài internet, đồng thời ép hàm API trả về một mã phản hồi thành công giả mạo (Fake SUCCESS). Điều này khiến ứng dụng tin rằng nó đã kết nối thành công với máy chủ và nhận được giấy phép hợp lệ.

1.5. Các công cụ hỗ trợ

1.5.1. Nhóm công cụ phân tích cấu trúc

Nhóm này được sử dụng trong giai đoạn "phân tích tĩnh" (không chạy trực tiếp chương trình). Chúng giúp kỹ sư kiểm tra cấu trúc của tệp tin (thường là PE trên Windows, ELF trên Linux, hoặc APK trên Android), trích xuất các siêu dữ liệu (metadata), xem các thư viện được gọi (Imports/Exports), tìm kiếm chuỗi văn bản (Strings), và xác định xem tệp có bị nén hay mã hóa (Packed/Obfuscated) hay không.

Các công cụ tiêu biểu:

- PE Studio: Công cụ tiêu chuẩn để phân tích tĩnh mã độc, giúp đánh dấu các yếu tố đáng ngờ (Indicators) trong cấu trúc file PE.
- CFF Explorer / PE-Bear: Trình chỉnh sửa và phân tích cấu trúc PE header rất trực quan, cho phép xem chi tiết từng section của file.



CFF Explorer

dành cho Windows

Miễn phí ★ 1 (1 ⭐) ↓ 300

✓ Chương trình đáng tin cậy V 7

Hình 1 : Công cụ CFF Explorer

- Binwalk: Chuyên dùng để phân tích và trích xuất cấu trúc của các tệp tin Firmware (IoT, Router).

```
Binwalk v2.1.2b
Craig Heffner, http://www.binwalk.org

Usage: binwalk [OPTIONS] [FILE1] [FILE2] [FILE3] ...

Signature Scan Options:
  -B, --signature          Scan target file(s) for common file signatures
  -R, --raw=<str>          Scan target file(s) for the specified raw data
  -A, --opcodes            Scan target file(s) for common executable opcodes
  -m, --magic=<file>       Specify a custom magic file to use
  -b, --dumb               Disable smart signature keywords
  -I, --invalid            Show results marked as invalid
  -x, --exclude=<str>      Exclude results that match <str>
  -y, --include=<str>      Only show results that match <str>

Disassembly Scan Options:
  -Y, --disasm             Identify the CPU architecture of a target file
  -T, --minsn=<int>        Minimum number of consecutive instructions
  -k, --continue           Don't stop at the first match
```

Hình 2 : Công cụ Binwalk

1.5.2. Nhóm công cụ dịch ngược (Disassemblers & Decompilers)

Nhóm này có nhiệm vụ dịch từ mã máy (Machine Code) dạng nhị phân 0 và 1 sang ngôn ngữ hợp ngữ (Assembly) để con người có thể đọc được (Disassembler), hoặc cao cấp hơn là khôi phục lại gần giống với mã nguồn gốc của các ngôn ngữ bậc cao như C, C++, C#, Java (Decompiler).

Các công cụ tiêu biểu:

- IDA Pro (Interactive Disassembler): hỗ trợ hầu hết các kiến trúc vi xử lý và có khả năng phân tích cực kỳ mạnh mẽ (bản Hex-Rays đi kèm hỗ trợ Decompile ra mã C).

- dnSpy / ILSpy: Chuyên dụng để dịch ngược các ứng dụng được viết bằng .NET (C#, VB.NET).



Hình 3 : Công cụ dnSpy

1.5.3. Nhóm công cụ trình gỡ rối (Debuggers)

Nhóm này được sử dụng cho "phân tích động" (Dynamic Analysis). Trình gỡ rối cho phép bạn chạy chương trình trong một môi trường được kiểm soát. Bạn có thể đặt các điểm dừng (Breakpoints), chạy từng dòng lệnh (Step in / Step over), theo dõi và can thiệp vào các giá trị trong thanh ghi (Registers), bộ nhớ (Memory) hoặc ngăn xếp (Stack) theo thời gian thực để xem cách phần mềm thực sự hoạt động.

Các công cụ tiêu biểu:

- x64dbg (và x32dbg): Trình gỡ rối mã nguồn mở phổ biến nhất hiện nay trên Windows, giao diện thân thiện, có hệ thống plugin phong phú và thường được dùng thay thế cho OllyDbg.
- WinDbg: Trình gỡ rối chính thức của Microsoft. Rất mạnh mẽ, đặc biệt là trong việc gỡ rối ở mức nhân (Kernel-mode debugging) như phân tích lỗi màn hình xanh (BSOD) hoặc Rootkit.
- Immunity Debugger: Hỗ trợ viết script bằng Python, rất được giới viết mã khai thác (Exploit Development) ưa chuộng.



Hình 4 : Công cụ Immunity Debugger